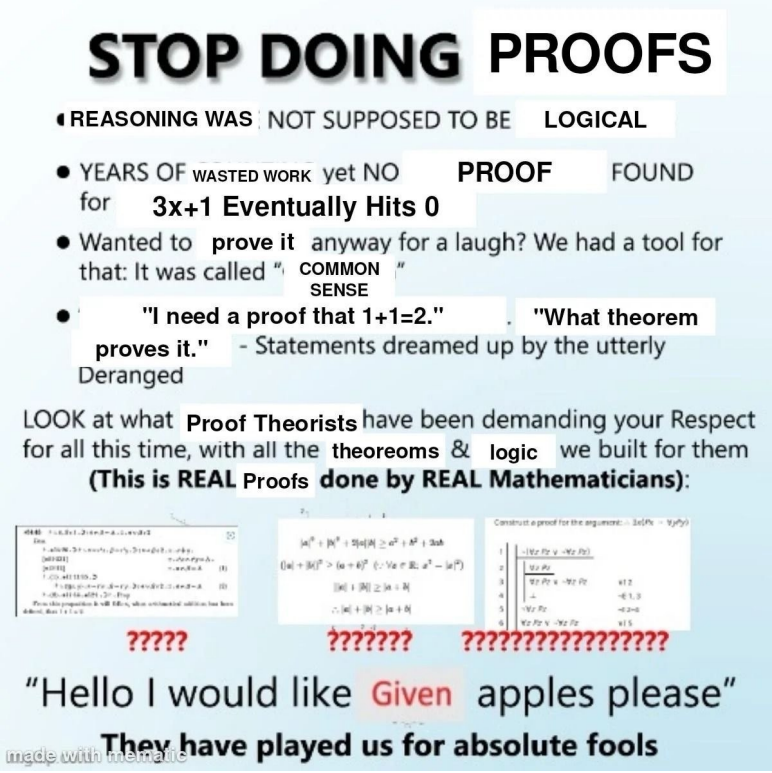
Technische Universität Wien

FORSYTE

Formal Methods in Systems Engineering

course material: slides



aglaia norza

October 17, 2026

norzaglaia@gmail.com
github.com/AglaiaNorza

Contents

1 Satisfiability in Formal Methods **2**
- 1.1 The SAT Decision Problem . . . 2
- 1.2 Formula transformation . . . 2
- 1.3 Resolution . . . 3
- 1.4 The DPLL Algorithm . . . 5
- 1.5 Clause-Driven Conflict Learning . . . 7
  - 1.5.1 Implication Graph . . . 7
  - 1.5.2 Clause Learning . . . 9
  - 1.5.3 Clause Database Reduction Strategies . . . 14
  - 1.5.4 Decision Heuristics . . . 14
  - 1.5.5 Restarts . . . 15
- 1.6 SAT-based solvers . . . 15
  - 1.6.1 Direct Approach (Non-Incremental, Non-Integrated) . . . 16
  - 1.6.2 Embedded Direct Approach (Non-Incremental, Integrated) . . . 17
  - 1.6.3 Incremental SAT Solvers . . . 17

2 Algebraic Reasoning **20**

3 Program Analysis **21**

4 Model Checking Approaches **22**

CHAPTER 1

Satisfiability in Formal Methods

1.1 | The SAT Decision Problem

> Def. 1 **The SAT decision problem**
>
> Given a propositional formula in conjunctive normal form, is there a truth assignment to the Boolean variables such that it satisfies all clauses?

SAT is an *NP-complete* problem, and it's a very active research area.

1.2 | Formula transformation

We have that

- every formula can be transformed into an **equivalent** CNF formula with, in the worst case, an exponential increase in size
- every formula can be transformed into an **equisatisfiable** formula with a *linear* increase in size (by introducing new variables for subformulae)

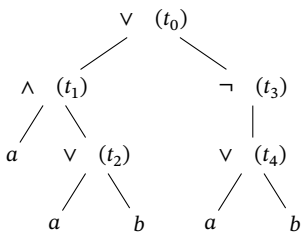
> **Tseitin's transformation**
>
> 1 | introduce a fresh boolean variable for every connective
>
> 2 | encode each equivalence as a set of clauses
>
> 3 | collect all constraints in a big conjunction and enforce the variable representing the entire formula to be true

Example. Consider the following propositional formula:

$$\Phi = (a \wedge (a \vee b)) \vee \neg(a \vee b)$$

First, we break the formula down into an Abstract Syntax Tree (AST) and identify every unique operation, and we then assign a new variable to each (non-atomic) subformula:

$$\begin{aligned} t_0 &\leftrightarrow (t_1 \vee t_3) \\ t_1 &\leftrightarrow (a \wedge t_2) \\ t_2 &\leftrightarrow (a \vee b) \\ t_3 &\leftrightarrow \neg t_4 \\ t_4 &\leftrightarrow (a \vee b) \end{aligned}$$



We translate each of these equivalences into standard CNF clauses via:

- AND $X \leftrightarrow (Y \wedge Z)$:
  $\equiv (\neg X \vee Y) \wedge (\neg X \vee Z) \wedge (X \vee \neg Y \vee \neg Z)$
- OR $X \leftrightarrow (Y \vee Z)$:
  $\equiv (\neg X \vee Y \vee Z) \wedge (X \vee \neg Y) \wedge (X \vee \neg Z)$
- NOT $X \leftrightarrow \neg Y$:
  $\equiv (\neg X \vee \neg Y) \wedge (X \vee Y)$

Finally, to assert that the entire formula Φ is true, we assert the root variable (t_0) as a unit clause, and join it with the CNF encodings of all the subformulae:

$$CNF(\Phi) = t_0 \wedge (t_0 \leftrightarrow (t_1 \vee t_3)) \wedge (t_1 \leftrightarrow (a \wedge t_2)) \wedge (t_2 \leftrightarrow (a \vee b)) \wedge (t_3 \leftrightarrow \neg t_4) \wedge (t_4 \leftrightarrow (a \vee b))$$

We get the fully expanded CNF formula:

$$\begin{aligned} CNF(\Phi) = t_0 \wedge\ & \\ & (\neg t_0 \vee t_1 \vee t_3) \wedge (t_0 \vee \neg t_1) \wedge (t_0 \vee \neg t_3) \wedge \\ & (\neg t_1 \vee a) \wedge (\neg t_1 \vee t_2) \wedge (t_1 \vee \neg a \vee \neg t_2) \wedge \\ & (\neg t_2 \vee a \vee b) \wedge (t_2 \vee \neg a) \wedge (t_2 \vee \neg b) \wedge \\ & (\neg t_3 \vee \neg t_4) \wedge (t_3 \vee t_4) \wedge \\ & (\neg t_4 \vee a \vee b) \wedge (t_4 \vee \neg a) \wedge (t_4 \vee \neg b) \end{aligned}$$

This resulting formula is equisatisfiable to the original Φ.

Plasted-Greenbaum's transformation

Every direction in the Tseitin encoding can be replaced by $\rightarrow$ or $\leftarrow$, where the direction of the implication depends on the *polarity* of the formula

- for every occurrence of a subformula φ, the polarity is $+$ if the number of negations above φ in the syntax tree is even, and $-$ otherwise
- we add $t_i \rightarrow$ if the polarity of t_i is positive, and $\leftarrow t_i$ otherwise
- so, every $\rightarrow$ and $\leftarrow$ can be replaced by at most two clauses

1.3 | Resolution

The theory underlying SAT solving is propositional resolution.

Def. 2 **Resolution**

The resolution rule is:

$$\frac{C \cup \{v\} \qquad D \cup \{\neg v\}}{C \cup D}$$

Which tells us that resolving the clause $C \cup \{v\}$ with the clause $D \cup \{\neg v\}$ results in the resolvant clause $D \cup C$.

We also have a **unit-resolution** rule (or **BCP**, boolean constraint propagation). This rule can

be used on so-called *unit clauses*, clauses with a single literal, which needs to be assigned to $\top$ in a CNF.

$$\frac{C \cup \{\neg v\} \qquad \{v\}}{C} \quad \{I, \neg I\} \cap C = \varnothing$$

> NB!!
>
> This rule is *not refutation-complete*, but it can be applied in time *linear* to the size of the formula.

With the unit-resolution rule, we can do **Unit Propagation**: when a unit resolution produces a unit clause, the unit-resolution step can be repeated with the new clause (until fixpoint).

BCP algorithm

- find a unit clause I
- remove all the clauses in which I occurs
- remove all literals $\neg I$ in clauses
- repeat

Example We want to check the validity of:

$$a \wedge b \rightarrow (a \leftrightarrow b)$$

To check validity, we must check unsatisfiability of the negated formula.

1 | CNF of negated formula:

$$\begin{aligned}
&\neg(a \wedge b \rightarrow (a \leftrightarrow b)) \\
\equiv\ & a \wedge b \wedge \neg(a \leftrightarrow b) \\
\equiv\ & a \wedge b \wedge \neg((\neg a \vee b) \wedge (a \vee \neg b)) \\
\equiv\ & a \wedge b \wedge (\neg(\neg a \vee b) \vee \neg(a \vee \neg b)) \\
\equiv\ & a \wedge b \wedge ((a \wedge \neg b) \vee (\neg a \wedge b)) \\
\equiv\ & a \wedge b \wedge (a \vee \neg a) \wedge (a \vee b) \wedge (\neg b \vee \neg a) \wedge (\neg b \vee b) \\
\equiv\ & a \wedge b \wedge (a \vee b) \wedge (\neg a \vee \neg b)
\end{aligned}$$

2 | Unit resolution on a with $(\neg a \vee \neg b)$

$$a \wedge b \wedge (a \vee b) \wedge (\neg a \vee \neg b) \wedge (\neg b)$$

3 | Unit propagation:

$$b \wedge (\neg b)$$

4 | Unit resolution on b with $(\neg b)$

$$\text{empty clause} \bot$$

The negation is UNSAT, therefore the original formula is valid.

1.4 | The DPLL Algorithm

The DPLL (Davis-Putnam-Logemann-Loveland) algorithm followed the DP (Davis-Putnam) algorithm. Where the first eliminated variables via full resolution, causing an explosion in memory, DPLL branches on variable values, trading memory for execution time (DPLL runs in polynomial space, at the cost of a worst case exponential runtime).

DPLL algorithm

- apply repeated unit resolution and removal of subsumed clauses

  at this point, any literals have been removed and the other clauses have been "cleaned up"
- if $\varnothing \in F$, return UNSAT
- if $F = \varnothing$ (no clauses left), return SAT
- choose a new variable x (using a decision heuristic), and branch on it:
  - add $\{x\}$ and recursively solve
  - add $\{\neg x\}$ and recursively solve
- if one of the branches returns SAT, return SAT
- otherwise, return UNSAT

There is an optional optimisation that can be added (but that could be too expensive compared to simple BCP):

Pure Literal Rule

If a variable appears only with positive polarity (or only negative polarity) throughout F:

- setting that literal to $\top$ cannot falsify any clause
- all clauses containing that pure literal can be eliminated immediately

Kinds of assignments in DPLL:

- **Decided ($\bigwedge$):** arbitrary choice/guess when BCP cannot proceed
- **Propagated (|):** values logically forced by unit resolution (BCP) until $\bot$ or a stall

The main drawback of the DPLL algorithm is its **blindness to found conflicts**:

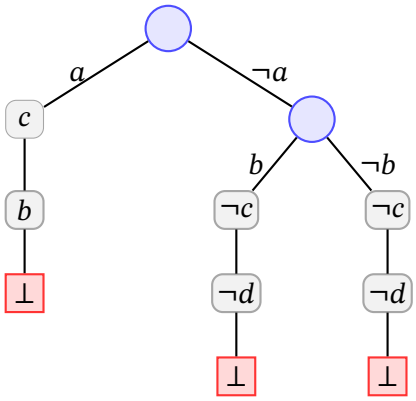
- when a conflict ($\bot$) is encountered, DPLL mechanically undoes only the most recent decision and tries the opposite branch, without analyzing which decisions actually caused the conflict (*chronological backtracking*)
- if a conflict is caused by an independent subset of variables, changing the assignment of an unrelated higher-level variable (like c) forces DPLL to rediscover and prove the exact same conflict all over again
- DPLL does not remember or record why a dead end was reached; it possesses no memory across distinct branches of the search tree

DPLL example: variable choosing Let's consider an example:

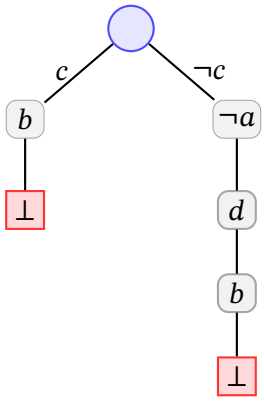
$$\mathcal{F} = (\neg b \vee \neg d) \wedge (b \vee \neg d) \wedge (\neg b \vee \neg c) \wedge (b \vee \neg c) \wedge (\neg a \vee c) \wedge (a \vee c \vee d)$$

Initially, no unit clauses exist, so the solver must decide on an unassigned variable.

The choice of an initial variable can heavily influence the search tree:



Tree 1: first decision on a
(requires 2 decision levels)



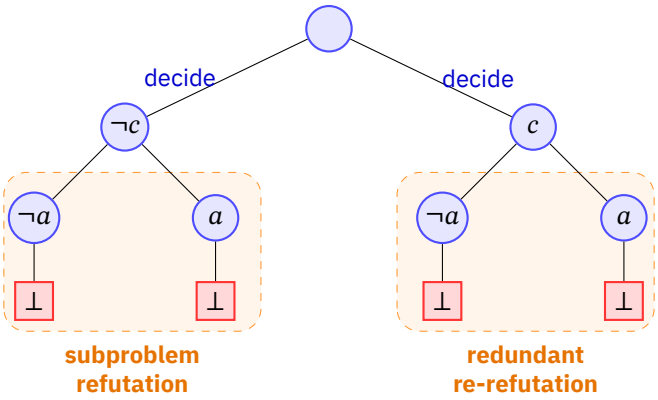
Tree 2: first decision on c
(no secondary decisions needed)

Figure 1.1: DPLL search trees for $\mathcal{F}$. Circles represent decision branch points; rounded rectangles represent literals forced via BCP; and red boxes represent conflict nodes ($\perp$).

DPLL example: blindness to found conflicts Consider the formula:

$$\mathcal{F} = (\neg c \vee a \vee b) \wedge (\neg a \vee \neg b \vee c) \wedge (\neg \mathbf{a} \vee \neg \mathbf{b}) \wedge (\neg \mathbf{a} \vee \mathbf{b}) \wedge (\mathbf{a} \vee \neg \mathbf{b}) \wedge (\mathbf{a} \vee \mathbf{b})$$

Assume the decision heuristic chooses literals in the order: $\neg c \prec c \prec \neg a \prec a \prec \neg b \prec b$.



- under $\neg c$, splitting on $\neg a$ and a yields contradictions because the subformula over a, b is unconditionally unsatisfiable
- chronological backtracking flips $\neg c \rightarrow c$.
- cnder c, DPLL has *no memory* of the previous derivation and blindly duplicates the exact same exploration on $\neg a$ and a

1.5 | Clause-Driven Conflict Learning

Clause-Driven Conflict Learning improves on DPLL by implementing **backjumping**, a technique that helps to recover from bad decisions by skipping redundant conflicts.

To implement backjumping, one needs to recognise, store and update *which decisions* contributed to conflicts. CDCL solvers achieve this through **Conflict Analysis** and **Clause Learning**.

1.5.1 | Implication Graph

An **implication graph** is a labelled DAG $G = (V, E)$ where

- V represents the literals of the current partial assignment (*trail*)

  nodes are labelled with their corresponding literal and decision level ($x = v@d$ where $v \in \{0, 1\}$)
- E represents the reasons of the assignments

  edges are labelled with the reason clause used for propagation
- there is a special node k representing a conflict - it has no outgoing edges, and its incoming edges are:

  $$\{(l, k) | \neg l \in C, C \in F\}$$

  - l is a literal node in the implication graph that has been assigned true
  - $C \in F$ is the clause that caused the conflict (i.e. has been falsified)
  - $\neg l \in C$ is the qualifying condition for the edge: the negation of l is a component of the clause C ($\neg l$ is false, and this falsification contributes to breaking clause C)
  - (l, k) a directed edge is drawn from the node l to the conflict node k, labeled with C to track which clause caused the conflict

> Example
>
> Suppose the formula contains the clause $C = (\neg x_1 \vee x_2 \vee \neg x_3)$.
>
> For C to become a conflicting (falsified) clause, the current decision trail must contain the assignments:
>
> - $x_1 =$ true (node x_1 exists in the graph)
> - $x_2 =$ false (node $\neg x_2$ exists in the graph)
> - $x_3 =$ true (node x_3 exists in the graph)
>
> The nodes responsible for the conflict are x_1, $\neg x_2$, and x_3. Therefore, the incoming edges to the conflict node κ will originate from x_1, $\neg x_2$, and x_3, representing the assignments that caused C to fail.

Implication Graph Example. Let the formula F be:

$$F = C_1 \wedge C_2 \wedge C_3 = (\neg a \vee \neg b \vee c) \wedge (\neg b \vee \neg c) \wedge (\neg a \vee b)$$

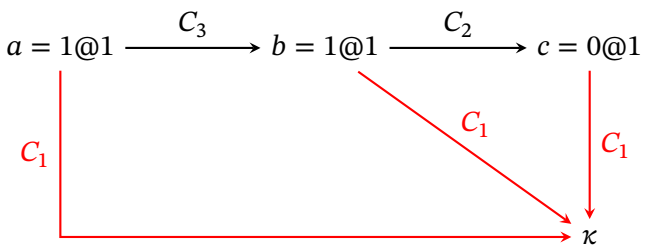
Suppose our first decision is to assign a to true (at decision level 1). This triggers Boolean Constraint Propagation (BCP) as follows:

1 | $a = 1$ forces $b = 1$ to satisfy C_3

2 | $b = 1$ forces $c = 0$ to satisfy C_2

3 | now $a = 1$, $b = 1$, and $c = 0$, which falsifies clause C_1

This propagation and resulting conflict can be visualized in the implication graph below:



Simple implementation details

In modern CDCL solvers, the implication graph is not stored as an explicit graph data structure. Instead, it is implicitly maintained via the assignment trail (stack of currently assigned variables).

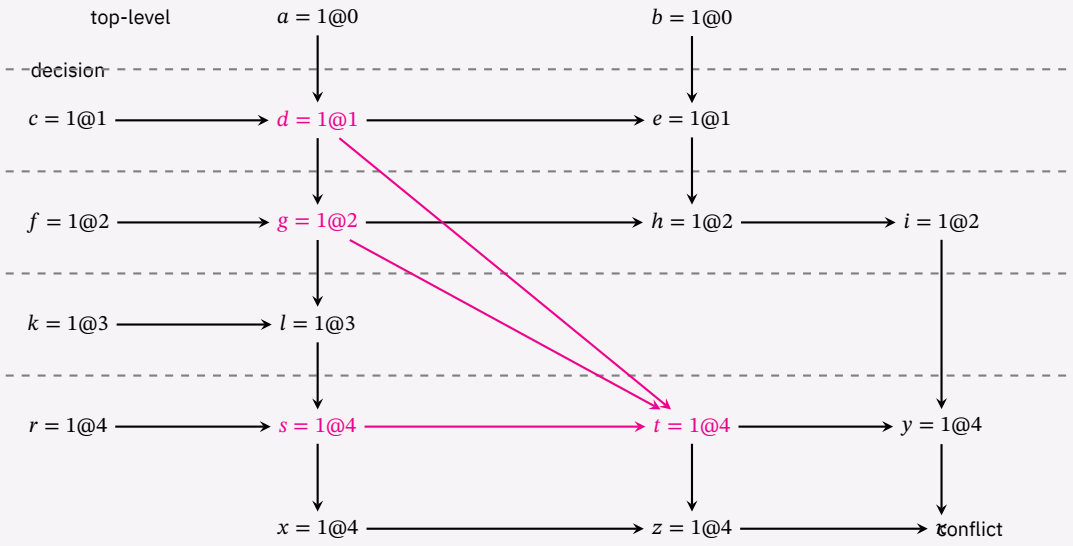
For each assigned variable, we store a tuple `(Value, Decision Level, Reason Pointer)`:

- **reason pointer (antecedent):** pointer to the clause that forced this assignment

  storing this pointer allows the solver to instantly trace backward from a conflict to compute a learned clause via resolution, without re-scanning the clause database

- **decision variables:** variables chosen by the decision heuristic (guesses) are not forced by any clause. Their reason pointer is set to `null` (so,during conflict analysis, a `null` pointer signifies a root node in the implication graph)

Exercise: how to read the graph



$$d \wedge g \wedge s \rightarrow t \quad \equiv \quad (\bar{d} \vee \bar{g} \vee \bar{s} \vee t)$$

Implication graphs allow us to trace the causes of variable assignments and conflicts.

1 | **Finding the cause of an assignment (the antecedent):**
- locate the node n whose assignment you want to analyze
- trace its incoming edges (look at the arrows pointing into n); let's say they come from nodes a, b, and c
- reconstruct the underlying clause: the edges imply that $a \wedge b \wedge c \rightarrow n$; converting this implication into standard CNF format gives us: $\neg a \vee \neg b \vee \neg c \vee n$

2 | **Finding the conflicting clause for a conflict node (κ):**
- the same logic applies, but the implication has no consequent (it implies a contradiction/false)
- trace the incoming edges to κ (e.g., from nodes x and y)
- simply negate these parent nodes and connect them with disjunctions; the clause that caused the conflict is $\neg x \vee \neg y$

1.5.2 | Clause Learning

To help us decide where to backjump to, and to learn more from a found conflict, we can add **clause learning** to our algorithm.

Whenever we encounter a conflict clause, we can add it to the formula: the literals contributing to a conflict form a partial assignment, which is just a conjunction of literals, whose negation is a clause that is *implied by the original CNF*. Therefore, adding this clause prevents the partial assignment from happening again during the search.

Finding cuts in the graph

A visual way to see this is to imagine it as finding cuts in the implication graph. We can draw a **cut** through the implication graph that separates the decision variables (roots) from the conflict node κ (leaf).

However, not all cuts are useful. There are two extremes we want to avoid:

- the decision cut (closest to roots)
  - results in a disjunction of all the negated decisions that led to the conflict
  - it is usually very long and provides almost no new information: it essentially just tells the solver, "don't make this exact combination of guesses again," which does not help prune the remaining search space efficiently
- the conflict cut (closest to leaf):
  - results in the exact clause that was just falsified to cause the conflict
  - gives no new information: the solver already has this clause in its original formula database

Because the decision cut is too broad (weak) and the conflict cut is redundant, modern CDCL solvers look for a cut in the middle of the graph. The goal is to find a cut that produces a **short** clause (because shorter clauses restrict the search space more heavily) and contains **novel information** not explicitly written in the original formula.

1.5.2.1 Unique Implication Points (UIPs) and Asserting Clauses

To find the optimal cut in an implication graph, modern solvers rely on finding a "bottleneck" that forces the conflict. This bottleneck is called a Unique Implication Point.

> Def. 3 **Unique Implication Point (UIP)**
>
> A node in the implication graph at the *current* decision level d is a UIP if *every path* from the decision variable at level d to the conflict node κ must pass through it.
>
> - in graph theory, a UIP is known as a **dominator** of the conflict node with respect to the decision node
> - (it represents a single, unavoidable consequence at the current decision level that triggered the conflict; if you remove the UIP, the conflict cannot happen)
>
> > Nota Bene
> >
> > The decision variable itself is *always* trivially a UIP, because all paths at the current level originate from it.

Because there can be multiple UIPs in a single implication graph (e.g., the decision variable, and several intermediate forced variables), we must choose for our cut.

The best one to choose is almost always the **First UIP**, i.e. the one *closest to the conflict node* k.

> Why the First UIP?
>
> - *Locality:* it captures the most recent, localized reasons for the conflict, making the resulting learned clause highly relevant to the current search space
> - *Efficiency:* the algorithm to find the 1UIP works backward from the conflict; stopping at the *first* UIP we encounter is computationally faster than resolving all the way back to the decision variable

When we perform resolution backward from the conflict and stop at the First UIP, the resulting learned clause is called an **Asserting Clause**.

> Def. 4 **Asserting Clause**
>
> A conflict clause that contains *exactly one literal from the current decision level* (which is the negation of the UIP node). All other literals in the clause come from earlier, lower decision levels.

This structural property of the asserting clause makes non-chronological backjumping work perfectly:

- when we learn an asserting clause, we look at the decision levels of its literals and backjump to the **second-highest decision level** present in the clause
- because the clause only had *one* literal from the highest level, backjumping clears that assignment
- upon arriving at the second-highest level, **every literal in the learned clause evaluates to False, except for one** (the UIP literal, which is now unassigned)

- this immediately triggers Boolean Constraint Propagation; the learned clause *asserts* the forced assignment of the UIP literal, driving the solver forward without needing a new blind guess

Detection of UIPs via Resolution

To compute the First UIP clause (the asserting clause), the solver iteratively computes (by resolution) the clauses that are associated with cuts moving *backward* from the conflict.

UIP Detection Algorithm

1 | let c_0 be the first conflict clause

2 | pick the literal $\ell \in c_0$ that was **last assigned** on the trail

3 | **while** there is more than one literal in c_i that is assigned at the *current* decision level:

  1 | let c_{i+1} be the resolvent of c_i and the *reason clause* of ℓ (apply a resolution step to c_i and the reason clause of ℓ)

  2 | let ℓ be the literal $\ell' \in c_{i+1}$ that was last assigned

- The literal ℓ in our conflict clause is false (which contributed to the conflict). Because ℓ was forced via BCP, it has a *reason clause* that caused its assignment.
- By resolving the current conflict clause C_i with the reason clause of ℓ, we eliminate ℓ (and $\neg\ell$) and merge the remaining literals. We have effectively taken a step backward in the implication graph, moving the "cut" behind ℓ (every intermediate generated clause C_i corresponds to a valid cut in the implication graph). To ensure we move backward through the graph without skipping or getting tangled, we resolve variables in the exact reverse order of how they were assigned.
- The loop stops exactly when C_i contains only *one* literal from the current decision level. The last C_i produced is the **First UIP-clause**
- As we established, because this final clause contains exactly one literal from the highest decision level, it is an asserting clause

First UIP Example

- **conflict node (κ):** falsified by assignments $y = 1@4$ and $z = 1@4$.
- **initial conflicting clause (c_0):** $(\neg y \vee \neg z)$

resolution trace:

1 | **1st resolution (resolving y):**
  - the literal y was forced by h, i (from level 2) and t (from level 4).
  - **reason for y:** $(\neg h \vee \neg i \vee \neg t \vee y)$
  - **resolvent (c_1):** resolve c_0 with reason(y) to eliminate y.
  - $c_1 = (\neg h \vee \neg i \vee \neg t \vee \neg z)$
  - *level 4 literals remaining:* t and z. (more than one, so we continue).

2 | **2nd resolution (resolving t):**
  - the literal t was forced by d (level 1), g (level 2), and s (level 4).

- **reason for t:** $(\neg d \vee \neg g \vee \neg s \vee t)$
- **resolvent (c_2):** resolve c_1 with reason(t) to eliminate t.
- $c_2 = (\neg d \vee \neg g \vee \neg s \vee \neg h \vee \neg i \vee \neg z)$
- *level 4 literals remaining:* s and z.

3 | **3rd resolution (resolving z):**

- the literal z was forced purely by x (level 4).
- **reason for z:** $(\neg x \vee z)$
- **resolvent (c_3):** resolve c_2 with reason(z) to eliminate z.
- $c_3 = (\neg x \vee \neg d \vee \neg g \vee \neg s \vee \neg h \vee \neg i)$
- *level 4 literals remaining:* x and s.

4 | **4th resolution (resolving x):**

- the literal x was forced by s (level 4).
- **reason for x:** $(\neg s \vee x)$
- **resolvent (c_4):** resolve c_3 with reason(x) to eliminate x.
- *note:* because $\neg s$ is already present in c_3, merging them performs *self-subsuming resolution* (the duplicate simply absorbs).
- $c_4 = (\neg d \vee \neg g \vee \neg s \vee \neg h \vee \neg i)$
- *level 4 literals remaining:* **only s!**

conclusion: c_4 now contains exactly one literal ($\neg s$) from the highest decision level (level 4), so the algorithm terminates.

- **first UIP:** the variable s.
- **the learned (asserting) clause:** $(\neg d \vee \neg g \vee \neg s \vee \neg h \vee \neg i)$.
- **backjumping execution:** the solver looks at the variables in the learned clause (s@4, d@1, g@2, h@2, i@2); the second-highest decision level is **level 2**, so the solver will backjump to it, clear all level 3 and 4 assignments, and the new learned clause will immediately force s to be false via BCP.

Minimizing the 1st UIP Clause

While the asserting clause is good enough, it can often be shrunk further. Shorter clauses often preferred, as they restrict the search space more effectively. Modern solvers use resolution techniques to analyze the implication graph and prune redundant literals from the learned clause.

there are two levels of minimization: **local** and **recursive**.

Local Minimization

Local minimization looks at the *immediate* antecedents (parents) of the literals in the learned clause.

- If all the immediate antecedents of a literal ℓ are *already present* (marked) in the learned clause, then ℓ is redundant and can be removed.

- This is an application of *self-subsuming resolution*. If our learned clause contains $(\ldots \vee \neg h \vee \neg i)$, and the reason i was forced in the first place was h (meaning the reason clause is $\neg h \vee i$), we can resolve them. Resolving the learned clause with the reason clause eliminates i and leaves us with $(\ldots \vee \neg h \vee \neg h)$, which simplifies to just $(\ldots \vee \neg h)$.

  (if i was only forced to be true because h was true (reason clause: $\neg h \vee i$), then h being true automatically drags i along with it; therefore, assigning $h =$ true is enough to guarantee the conflict by itself, and we do not need to explicitly ban i: banning h covers the root cause)

  (we resolve the learned clause $(\ldots \vee \neg h \vee \neg i)$ with the reason clause $(\neg h \vee i)$, which yields $(\ldots \vee \neg h)$, safely eliminating the redundant variable i)

Algorithm:

```
1 | mark all variables present in the first UIP clause
2 | for each literal, check its reason clause
3 | if every other literal in the reason clause is also marked, remove the candidate literal
```

Recursive Minimization

Local minimization is fast but misses deeper redundancies. What if a literal's antecedent is *not* in the learned clause, but that antecedent was forced by things that *are*? Recursive minimization traces paths all the way back through the graph.

- a candidate literal can be removed if *every path* backward from its antecedents in the implication graph eventually terminates at either:

  ```
  1 | a literal that is already marked (present in the learned clause)
  2 | a top-level assignment (decision level 0), because level 0 variables are universally true
  ```

Algorithm:

```
1 | mark all variables in the 1st UIP clause
2 | for each candidate literal, DFS backward through its antecedents in the implication graph
3 | if the search always hits marked literals or level 0 assignments, the candidate is redundant and is removed
```

- To keep this fast, if the backward search ever hits an unmarked decision variable, or pulls in a variable from a different decision level (that isn't level 0), the search for that candidate terminates early with a failure (the candidate cannot be removed).

Example

Consider the first UIP:

$$(\neg d \vee \neg g \vee \neg s \vee \neg h \vee \neg i)$$

- *local minimization:* i is removed because its only antecedent is h, which is already in the clause; the clause becomes $(\neg d \vee \neg g \vee \neg s \vee \neg h)$.
- *recursive minimization:* h is investigated; its antecedents are g (which is marked) and e

(unmarked)

the search recursively checks e: the antecedents of e are d (marked) and b (Level 0); because all paths hit marked or Level 0 nodes, h is also safely removed

- *minimized clause:* $(\neg d \vee \neg g \vee \neg s)$

1.5.3 | Clause Database Reduction Strategies

While learning clauses is essential for pruning the search space, keeping *all* learned clauses forever is detrimental.

The main reasons why periodically cleaning out the clause database is beneficial are intuitive:

- too many clauses will eventually exhaust system memory
- the more clauses the solver has to watch and evaluate, the slower BCP becomes
- as the solver moves to different areas of the search space, older learned clauses often become irrelevant to the current search focus
- some clauses become permanently useless (e.g., if a clause is satisfied by a top-level assignment, it will never be falsified again)

Commonly-used heuristics include:

- *Size Heuristics:* shorter clauses are more powerful, so long clauses get discarded
- *Least Recently Used (LRU):* the solver tracks when a clause was last used in BCP or conflict analysis and deletes those that haven't been used recently (like in cache design)
- *Clause Activity Scores:* clauses can be given a score that is bumped every time they participate in generating a new conflict clause; low-scoring clauses are discarded
- **Glucose Level / LBD (Literal Block Distance)** (very common):
  - based on the number of *distinct decision levels* present in the literals in the learned clause
  - a clause might be long, but if all its variables were assigned at the same decision level, they are tightly clustered in the search tree; a low LBD means the clause acts like a short clause

1.5.4 | Decision Heuristics

Another important aspect is that of choosing which unassigned variable BCP should choose next.

A good heuristic should be fast to compute, it should lead the solver to conflicts quickly, and it should be adaptable to the solver's current focus (e.g. involving variables involved in recent conflicts).

VSIDS: Variable State Independent Decaying Sum

VSIDS is the standard heuristic used in almost all modern CDCL solvers. It scores variables based on their involvement in recent conflicts:

- every variable starts with a score (often 0, or based on its initial frequency in the formula)
- whenever a conflict occurs and a new clause is learned, the solver increases the score of all variables involved in the conflict analysis (e.g., variables in the learned clause and the paths resolving to it)

- periodically, the solver divides the scores of *all* variables in the system by a constant factor (so that the numbers don't take up too much space in memory / don't overflow)

When a decision has to be made, the solver simply picks the unassigned variable with the highest current score

1.5.5 | Restarts

In SAT solving, algorithms often suffer from "heavy-tail behavior.": due to a few bad early decisions, the solver might wander into a massive, heavily constrained subtree of the search space that takes too long to evaluate, even though a satisfying assignment (or a quick proof of unsatisfiability) might exist in a different branch.

To escape these dead ends, solvers frequently **restart** the search process.

- the solver abandons its current assignment trail and jumps completely back to Decision Level 0.
- but it doesn't reset its state completely: it keeps all the learned clauses, and the current VSIDS variable scores

Since the solver retains its learned clauses and heuristic scores, the next search will not be identical to the previous one:

- the VSIDS scores have been altered by the conflicts encountered during the abandoned run, meaning the solver will choose variables in a different order
- the newly learned clauses will act as roadblocks, steering the solver away from the bad paths that caused the previous conflicts

Restarts are scheduled according to various mathematical sequences to balance short, aggressive searches with deeper explorations.

1.6 | SAT-based solvers

There are different ways to encode problems into propositional logic, and there are different ways to interact with the solver.

We will analyse four different structures of SAT-based tools:

1 | direct approach: non-incremental non-integrated

2 | embedded direct approach: non-incremental integrated

3 | incremental integrated (IPASIR-based CEGAR)

4 | interactive integrated (IPASIR-UP External Propagators)

Example: 2-bit counter

Let us model a 2-bit counter with bits a (LSB) and b (MSB).

- initial state $I(s_0)$:

$$I(b, a) = \neg b \wedge \neg a$$

(counter starts at 0)

- transition relation $\mathcal{T}(s, s')$:

$$\mathcal{T}((b, a), (b', a')) = (b' \leftrightarrow (a \oplus b)) \wedge (a' \leftrightarrow \neg a)$$

(the counter increments by 1 modulo 4)

Let's say that we want to check if the counter ever reaches the value 2 (binary 10):

- we introduce a bad state $B(b, a) = b \wedge \neg a$

Our goal is to prove that the counter's value is always $\neq 2$.

This can be posed as a satisfiability problem: we need to find a (k-long) execution of the system where the counter has value 2 (a counterexample).

Such an execution trace exists if the following propositional formula is satisfiable for a specific bound k:

$$I(s_0) \wedge \mathcal{T}(s_0, s_1) \wedge \cdots \wedge \mathcal{T}(s_{k-1}, s_k) \wedge B(s_k)$$

We iteratively check this formula by increasing k:

- if the formula is UNSAT, the bad state is not reachable in k steps - we increase k again
- if the formula is SAT, the property is violated - the satisfying assignment is our counterexample

Let's unroll the execution for the first few bounds to see if the property is violated:

1 | $k = 0$: can the initial state violate the property?

$$I(s_0) \wedge B(s_0)$$

$$(\neg b_0 \wedge \neg a_0) \wedge (b_0 \wedge \neg a_0)$$

This is UNSAT.

2 | $k = 1$: can we reach the bad state in exactly one step?

$$I(s_0) \wedge \mathcal{T}(s_0, s_1) \wedge B(s_1)$$

$$(\neg b_0 \wedge \neg a_0) \wedge ((b_1 \leftrightarrow (a_0 \oplus b_0)) \wedge (a_1 \leftrightarrow \neg a_0)) \wedge (b_1 \wedge \neg a_1)$$

This evaluates to UNSAT.

3 | $k = 2$: can we reach the bad state in exactly two steps?

$$I(s_0) \wedge \mathcal{T}(s_0, s_1) \wedge \mathcal{T}(s_1, s_2) \wedge B(s_2)$$

This will return SAT, proving the counter reaches value 2 after two transitions.

1.6.1 | Direct Approach (Non-Incremental, Non-Integrated)

In the direct approach, the tool translates the entire Bounded Model Checking (BMC) problem for a specific bound k into a standard text file format (DIMACS).

For each bound k:

1 | Encode the complete BMC problem into a single SAT problem in DIMACS format

2 | Call a standalone SAT solver on the produced file

3 | Parse and evaluate the output of the solver to map it back to a counterexample

If the formula is UNSAT, the process repeats for $k+1$. The solver retains no memory of the previous bounds, meaning the formula must be constructed entirely from scratch every time.

DIMACS format

```
c DIMACS example
p cnf 5 3
1 -5 4 0
-1 5 3 4 0
-3 -4 0
```

- comments start with `c`
- the header has the format `p cnf m n` with `m` maximum variable index, `n` number of clauses
- one line per clause, terminated by a `0`: literals are encoded as signed integers

1.6.2 | Embedded Direct Approach (Non-Incremental, Integrated)

To eliminate the overhead of file I/O and parsing, the SAT solver can be embedded directly within the toolchain.

- uses the interface of the solver directly instead of DIMACS files
- the solving process itself remains non-incremental (the solver still solves each bound as a completely new, independent problem)

Downsides of direct approaches

While the simple solution can often be the best one, complete encodings for large bounds can become extremely large or even impossible to generate. Furthermore, not every part of the unrolled formula is actually relevant to finding a refutation.

1.6.3 | Incremental SAT Solvers

Instead of rebuilding the formula from scratch for every bound, Incremental SAT solving evaluates a sequence of related formulae $(F_0, F_1, F_2 \dots)$ by reusing the same solver instance.

Advantages

- smaller initial encoding overhead
- reuses previous reasoning steps: retains learned clauses, gathered information (phases, scores), and applied formula simplifications
- supported by the standardized IPASIR (Re-entrant Incremental Satisfiability API) interface, making different solvers interchangeable

The sequence of related formulas $(F_0, F_1, F_2, \dots)$ represents an **incremental building** of the overall problem. The solver retains the previous clauses and simply appends the new ones (Δ_i):

- $F_0 = \Delta_0$

- $F_1 = \Delta_0 \wedge \Delta_1$
- $F_2 = \Delta_0 \wedge \Delta_1 \wedge \Delta_2$

In Bounded Model Checking, Δ_i represents the newly unrolled transition relation $\mathcal{T}(s_{i-1}, s_i)$.

1.6.3.1 Assumptions

> Def. 5 **Assumptions**
>
> To manage incremental queries, we use **assumptions**: temporary unit clauses passed to the solver before a solve call.
>
> - assumptions act as forced decisions for the current SAT call.
> - they are temporary: once the solve call finishes, the assumptions disappear
> - if a query fails under a set of assumptions, the solver returns a subset of the assumptions that were responsible for the conflict (extracted via the learned clause of the very last conflict analysis)

Assumptions can also be used to "delete" or deactivate clauses:

1 | extend temporary clauses with an activation variable a: $(a \vee l_1 \vee \cdots \vee l_n)$.

2 | solve the formula under the assumption of $\neg a$; this forces the clause to be satisfied by one of the l_i literals (activating the clause)

3 | once the clause is no longer needed, add a permanent unit clause (a). Boolean Constraint Propagation (BCP) on the root-level will trivially satisfy the clause, and the next garbage collection can delete it (NB: can impact solver performance)

Example. Consider the formula:

$$F = (a \vee b) \wedge (a \vee \neg c) \wedge (\neg b \vee d)$$

We can query the incremental solver with temporary assumptions:

1 | *query 1:* "is F satisfiable?"
- *result:* SAT (e.g., $a = \top, b = \top, c = \bot, d = \top$)

2 | *query 2:* "is F satisfiable assuming specific literals (e.g., $\neg a, \neg b, c$) are true?"
- *result:* UNSAT; $a = \bot$ forces $b = \top$ via the first clause, which conflicts with $b = \bot$
- *UNSAT core:* the solver returns the specific subset of assumptions responsible for the conflict (e.g., $\{\neg a, \neg b\}$)

3 | *query 3:* "Is F satisfiable assuming c is true?"
- *result:* SAT;

> Incremental BMC Example
>
> Using the 2-bit counter:

```
partially encode the initial state I(s_0)
assume the bad state B(s_0) and solve; if unsat, extract the core
add the transition relation 𝒯(s_0, s_1) permanently
assume the bad state B(s_1) and solve again
repeat until the solver returns SAT
```

1.6.3.2 Handling of assumptions

CHAPTER 2

Algebraic Reasoning

CHAPTER 3

Program Analysis

CHAPTER 4

Model Checking Approaches